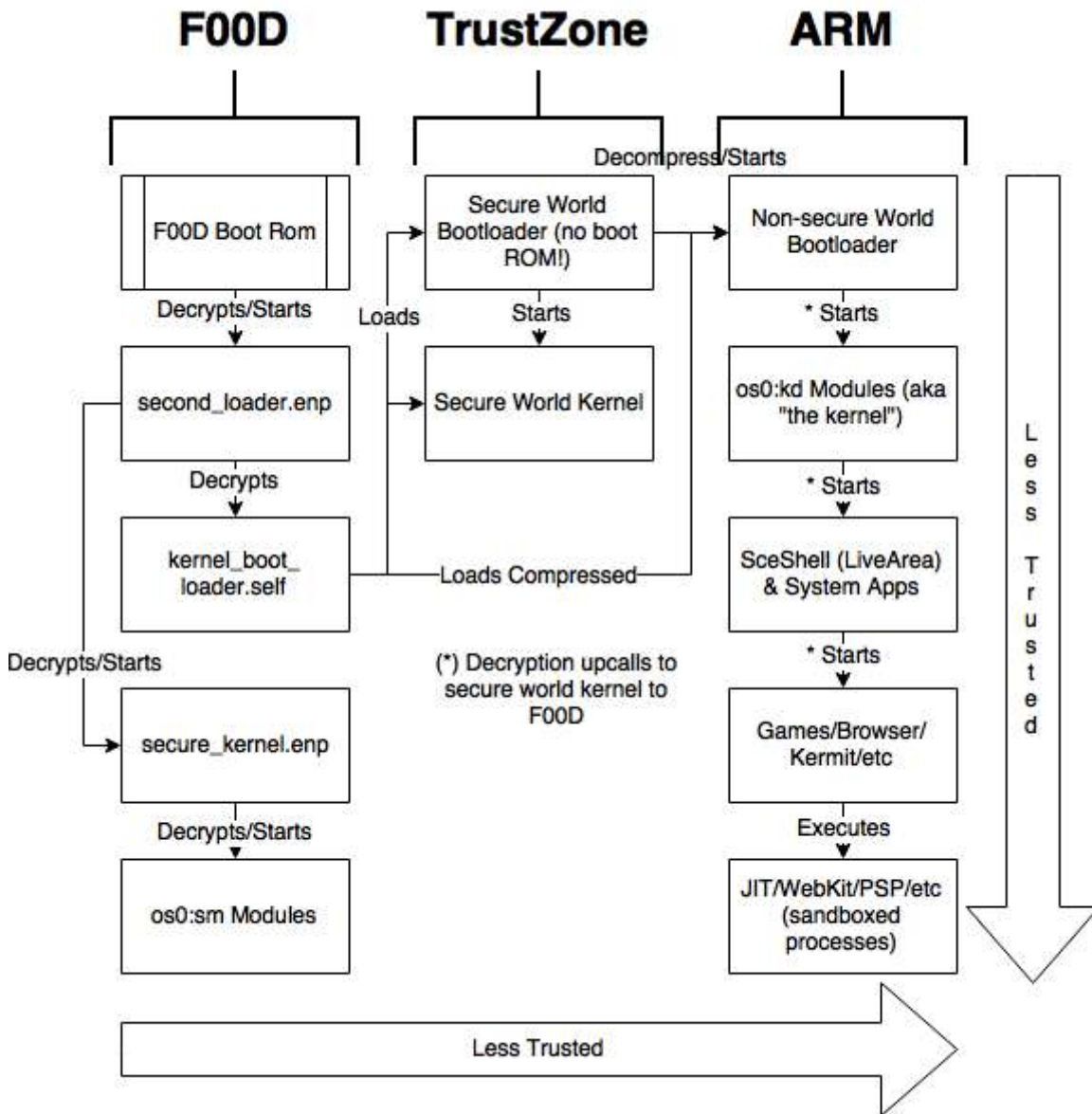


Security



Right and down are less trusted. `kernel_boot_loader.self` does not itself run on cmep but only contains encrypted segments for ARM to run. Unless indicated otherwise, all processes are encrypted and require cmep to decrypt. This diagram reflects current knowledge and may contain inaccuracies (specifically everything on the cmep side is only a best guess).

Contents

Root Chain of Trust

Algorithm

Boot FW version check

Decryption Request

Security

Usermode

Non-secure Kernel

ARM Secure Kernel

Secure/non-secure Kernel Bootloader

Cmep Secure Modules

cMeP Secure Kernel

Second Loader

First Loader (Boot ROM)

Root Chain of Trust

The root is Cmep's bootrom (First Loader). This is where root keys are seeded and wiped from memory. `second_loader.enp` is loaded from the eMMC (SLB2 partition) or SD Card (if SD BOOT challenge passes). Both `second_loader.enp` and `secure_kernel.enp` files are per-console encrypted from their respective `.enc` files (by cmep) during the update process. This is in addition (wraps over) to the already encrypted `.enc` from the update file and is to hinder downgrade attacks (by externally flashing to eMMC a bootloader from another PS Vita).

Algorithm

Algorithm	Used by	Comments
ECDSA160	<u>SceNpDrm</u> files (act.dat, RIF)/PKG header/some PSP files	With SCE pointbase
ECDSA224	<u>SceMsif</u> / <u>SceNpDrm</u> files (?)	With SCE pointbase
ECDSA256	<u>SceKrm</u>	With <code>ecp_nistz256</code> (it looks like an algorithm that speeds up <code>secp256r1</code>)
RSA2048	<u>SceNpDrm</u> / <u>SceAppMgr</u> /qaf token/Kit activation file/Certified File/ <u>SceNpDrm</u> files (act.dat, RIF)	N/A
AES 128/192/256	In too many places to list.	N/A
DES/3DES	<u>SceMsif</u>	For rmath
SHA 1/224/256	In too many places to list.	N/A
PRNG	<u>SceKernelModulemgr</u>	For ASLR
TRNG	<u>SceMsif</u> / <u>SceSblSsMgr</u>	N/A
base64	<u>SceNpCommon</u>	N/A

Boot FW version check

IdStorage SMI leaf. (If `smi_fw` > `second_loader_fw_version`, error out)

There are two FW version checks in SNVS. (if `snvs_fw` != `second_loader_fw_version`, error out)

FW version check in kernel_boot_loader.self. (if is_resume == 0 && kbl_fw_version != second_loader_fw_version, error out)

Maybe more?

Decryption Request

If something needs to be decrypted from a user/system application (PVF, SELF or PUP for example), first the request is passed to the non-secure kernel. The non-secure kernel does some checks (mostly integrity checks) then makes a request to TrustZone to load a new Secure Modules (to have the right keys and the right program to parse the data format). Next the non-securekernel passes the data to the secure kernel to decrypt and the secure kernel passes it to cmep. Cmep does the real checks (integrity, signature, format, etc) and decrypts the data directly into the shared DRAM. After clearing the cache, the non-secure kernel has access to the data. Cmep will not decrypt if anything fails in its checks.

Security

Since most decryption happens on cmep and a signed revoke list is passed early during startup (in TrustZone modules initialization), it is possible to revoke just about anything on the system. Any hack that can happen before the revoke list is loaded would likely count as a "permanent hack" that cannot be patched without a hardware revision.

Usermode

If a game/application/system application is hacked, Sony can easily distribute a firmware update to block the hack. They can also ask a game update and depending on the game that is hacked, they could enforce an update and therefore stop the exploit from triggering.

There are different sandboxes for applications, the more trusted being SceShell modules, which consist of the home screen, LiveArea, background menu, and background usermode tasks.

Non-secure Kernel

A hack at this level allows for running unsigned code and also pirated games. Decryption must be done through the secure kernel and cmep, so attempts at loading SELFs from past and future firmwares would not work. To counter piracy, Sony can encrypt new games with new keys and prevent them from running on old compromised firmwares.

ARM Secure Kernel

Same level of access and same results as non-secure kernel. Direct access to cmep interrupts could allow for a cmep attack though but there is little gain compared to simply having non-secure kernel code execution.

Secure/non-secure Kernel Bootloader

Allows for "non-tethered" hacks. Can be used to load unsigned kernel at an early stage. It does not allow getting more keys than Kernel hacks allowed.

Cmep Secure Modules

All keys are stored in cmep Secure Modules SELFs, which are validated and decrypted just like ARM SELFs. If Secure Modules are compromised, keys can be extracted to decrypt updates, bootloader, kernel SELFs, usermode SELFs, and more.

Sony can revoke most keys by updating slb2:prog_rvk.srvk, keys in SM SELFs, ENC files keys, etc... But SPKG key cannot be changed else older and current firmwares PUPs would not be installable anymore... Once SPKGs are decrypted, filesystem is accessible, but that does not mean Certified Files it embeds are decryptable because keys can change.

cMeP Secure Kernel

Secure Kernel has ability to freely access cmep and ARM memory, so a hack would allow dumping SMs i.e keys. It so has complete control over cmep and ARM. A hack would also allow to read the content of Cmep keyrings for which some embed keys.

Second Loader

Compare to Secure Kernel, Second Loader has access to more Cmep keyrings because they are not locked yet so a hack would allow to get a few more keys. However at the time Second Loader runs, Secure Kernel and ARM kernel bootloaders nor kernel are not in memory yet so a Second Loader hack would have to play with Bigmac keyrings or to run Secure Kernel itself in order to get code execution on lower levels.

First Loader (Boot ROM)

Boot rom handles the master keys so once it is hacked, most static derived key are computable once for all. As is not updatable (ROM) a First Loader hack is not patchable. However it is currently impossible to hack First Loader on final revision units without hardware techniques.

Retrieved from "<http://wiki.henkaku.xyz/index.php?title=Security&oldid=20413>"

This page was last edited on 4 August 2023, at 08:27.

Content is available under Creative Commons Attribution unless otherwise noted.